

Week 2: Ingest and Explore BTS T-100 Dataset

===== WEEK 2 START =====

Objective: Ingest BTS T-100 Domestic Segment (All Carriers) & perform dataset exploration.

1. Environment, project paths, and reusable functions

- All ingestion/exploration functions used below are included in the notebook.
- Same functions also in `Src/week2_src_t100_ingest_explore.py`.

```
In [1]: from pathlib import Path
import sys
from IPython.display import display

# Detect the repository root whether this notebook is launched from the root or Notebooks //.
START = Path.cwd().resolve()
PROJECT_ROOT = START if (START / "Data").exists() else START.parent
RAW_DIR = PROJECT_ROOT / "Data" / "raw"
REPORTS_DIR = PROJECT_ROOT / "Reports"

print(f"Project root: {PROJECT_ROOT}")
print(f"Raw data directory: {RAW_DIR}")
print(f"Reports directory: {REPORTS_DIR}")
```

Week 2 Functions

```
In [2]: """Week 2 ingestion and exploration utilities for BTS T-100 Domestic Segment data.

Module limited to Week 2 objectives:
1) Ingest annual BTS ZIP extracts,
2) Identify/construct the model target & candidate predictors,
3) Explore Dataset (schema, coverage, missingness, cardinality), and descriptive statistics,
4) Make tables/figures of report/code-output for Week 2.

"""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Iterable
from zipfile import ZipFile

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

ARCHIVE_GLOB = "week1_data_t100_domestic_segment_all_carriers_*.zip"

# Each set contains alias observed in BTS downloads / documents.
FIELD_ALIASES: dict[str, set[str]] = {
    "Year": {"YEAR"},
    "Quarter": {"QUARTER"},
    "Month": {"MONTH"},
    "UniqueCarrier": {"UNIQUECARRIER", "UNIQUECARRIERCODE"},
    "AirlineID": {"AIRLINEID"},
    "UniqueCarrierName": {"UNIQUECARRIERNAME"},
    "OriginAirportID": {"ORIGINAIRPORTID"},
    "Origin": {"ORIGIN"},
    "OriginCityName": {"ORIGINCITYNAME"},
    "OriginState": {"ORIGINSTATE"},
    "DestAirportID": {"DESTAIRPORTID", "DESTINATIONAIRPORTID"},
    "Dest": {"DEST", "DESTINATION"},
    "DestCityName": {"DESTCITYNAME", "DESTINATIONCITYNAME"},
    "DestState": {"DESTSTATE", "DESTINATIONSTATE"},
    "AircraftType": {"AIRCRAFTTYPE"},
    "AircraftConfig": {"AIRCRAFTCONFIG", "AIRCRAFTCONFIGURATION"},
    "Class": {"CLASS", "SERVICECLASS"},
    "Passengers": {"PASSENGERS"},
    "Seats": {"SEATS"},
    "DepScheduled": {"DEPSCHEDULED", "DEPARTURESCHEDULED"},
    "DepPerformed": {"DEPPERFORMED", "DEPARTURESPERFORMED"},
    "Distance": {"DISTANCE"},
    "LoadFactor": {"LOADFACTOR"},
}

REQUIRED_FIELDS = {
    "Year",
    "Month",
    "UniqueCarrier",
    "Origin",
```

```

        "Dest",
        "Class",
        "Passengers",
        "Seats",
        "DepScheduled",
        "DepPerformed",
        "Distance",
    }

NUMERIC_FIELDS = [
    "Year",
    "Quarter",
    "Month",
    "AirlineID",
    "OriginAirportID",
    "DestAirportID",
    "AircraftType",
    "AircraftConfig",
    "Passengers",
    "Seats",
    "DepScheduled",
    "DepPerformed",
    "Distance",
    "LoadFactor",
]

BTS_DESCRIPTIONS = {
    "DepScheduled": "Departures scheduled",
    "DepPerformed": "Departures performed",
    "Seats": "Available seats",
    "Passengers": "Non-stop segment passengers transported",
    "Distance": "Distance between airports (miles)",
    "LoadFactor": "Ratio of passenger miles to available seat miles",
    "UniqueCarrier": "Unique carrier code; preferred for analysis across years",
    "AirlineID": "DOT airline identification number",
    "OriginAirportID": "DOT origin airport identifier",
    "Origin": "Origin airport code",
    "DestAirportID": "DOT destination airport identifier",
    "Dest": "Destination airport code",
    "Year": "Calendar year",
    "Quarter": "Calendar quarter",
    "Month": "Calendar month",
    "Class": "Service class; core project scope uses F = scheduled passenger/cargo service",
}

CANDIDATE_PREDICTOR_RATIONALE = {
    "Passengers": "Most recent observed demand at forecast origin; strong persistence/level signal.",
    "Seats": "Most recent available capacity; proxy for planned supply and market size.",
    "DepScheduled": "Most recent scheduled frequency; captures offered service intensity.",
    "DepPerformed": "Most recent realized frequency; captures actual service delivered.",
    "LoadFactor": "Capacity utilization measure; may indicate supply-demand pressure.",
    "Distance": "Stable route characteristic associated with market type and service pattern.",
    "UniqueCarrier": "Carrier-specific network, fleet, and commercial effects.",
    "Origin": "Origin-market identity; captures location-specific demand structure.",
    "Dest": "Destination-market identity; captures location-specific demand structure.",
    "Year": "Long-run trend/regime information.",
    "Quarter": "Broad seasonal pattern.",
    "Month": "Fine-grained calendar seasonality.",
}

@dataclass
class Week2Results:
    monthly: pd.DataFrame
    dataset_profile: pd.DataFrame
    class_distribution: pd.DataFrame
    missingness: pd.DataFrame
    numeric_summary: pd.DataFrame
    variable_profile: pd.DataFrame
    annual_summary: pd.DataFrame
    top_routes: pd.DataFrame

def _norm(name: str) -> str:
    """Normalize a column name for robust schema matching."""
    return "".join(ch for ch in str(name).upper() if ch.isalnum())

def detect_project_root(start: Path | None = None) -> Path:
    """Locate the repository root from the current directory or a notebook subfolder."""
    start = (start or Path.cwd()).resolve()
    candidates = [start, *start.parents]
    for path in candidates:
        if (path / "Data").exists() and (path / "README.md").exists():
            return path
    # A forgiving fallback makes the notebook usable before README is created.
    for path in candidates:
        if (path / "Data").exists() and (path / "Notebooks").exists():
            return path
    raise FileNotFoundError(
        "Could not locate project root. Run from the repository root or Notebooks/ folder."
    )

```

```

def discover_archives(raw_dir: Path) -> list[Path]:
    """Return annual T-100 ZIP archives in deterministic filename order."""
    archives = sorted(raw_dir.glob(ARCHIVE_GLOB))
    if not archives:
        archives = sorted(raw_dir.glob("*.zip"))
    if not archives:
        raise FileNotFoundError(f"No ZIP archives found in {raw_dir}")
    return archives

def _data_member(zip_path: Path) -> str:
    """Choose the largest CSV/TXT member in a BTS archive."""
    with ZipFile(zip_path) as zf:
        candidates = [
            info
            for info in zf.infolist()
            if not info.is_dir() and info.filename.lower().endswith((".csv", ".txt"))
        ]
    if not candidates:
        raise ValueError(f"No CSV/TXT member found inside {zip_path.name}")
    return max(candidates, key=lambda x: x.file_size).filename

def _resolve_columns(columns: Iterable[str]) -> dict[str, str]:
    """Map canonical field names to actual BTS download headers."""
    normalized = {_norm(col): col for col in columns}
    resolved: dict[str, str] = {}
    for canonical, aliases in FIELD_ALIASES.items():
        for alias in aliases:
            if alias in normalized:
                resolved[canonical] = normalized[alias]
                break
    missing = REQUIRED_FIELDS.difference(resolved)
    if missing:
        raise ValueError(
            "Required T-100 fields were not found: "
            f"{sorted(missing)}. Available headers include: {list(columns)[:30]}"
        )
    return resolved

def inspect_archive_schema(zip_path: Path) -> tuple[str, pd.DataFrame]:
    """Read only the header of one archive and return the resolved Week 2 schema."""
    member = _data_member(zip_path)
    with ZipFile(zip_path) as zf, zf.open(member) as fh:
        header = pd.read_csv(fh, nrows=0)
        resolved = _resolve_columns(header.columns)
        schema = pd.DataFrame(
            [{"CanonicalField": k, "SourceHeader": v} for k, v in resolved.items()]
        )
    return member, schema

def _coerce_types(chunk: pd.DataFrame) -> pd.DataFrame:
    """Apply conservative type coercion after canonical renaming."""
    out = chunk.copy()
    for col in NUMERIC_FIELDS:
        if col in out.columns:
            out[col] = pd.to_numeric(out[col], errors="coerce")
    for col in [
        "UniqueCarrier",
        "UniqueCarrierName",
        "Origin",
        "OriginCityName",
        "OriginState",
        "Dest",
        "DestCityName",
        "DestState",
        "Class",
    ]:
        if col in out.columns:
            out[col] = out[col].astype("string").str.strip()
    return out

def _partial_monthly_aggregate(scheduled_chunk: pd.DataFrame) -> pd.DataFrame:
    """Aggregate one scheduled-passenger chunk to the project unit of analysis."""
    key_cols = [
        "Year",
        "Month",
        "UniqueCarrier",
        "Origin",
        "Dest",
    ]
    optional_identity = [
        "AirlineID",
        "UniqueCarrierName",
        "OriginAirportID",
        "OriginCityName",
        "OriginState",
        "DestAirportID",
        "DestCityName",
        "DestState",
    ]

```

```

present_identity = [c for c in optional_identity if c in scheduled_chunk.columns]

work = scheduled_chunk.copy()
for col in ["Passengers", "Seats", "DepScheduled", "DepPerformed", "Distance"]:
    if col not in work:
        work[col] = np.nan

group_cols = key_cols + present_identity
agg = (
    work.groupby(group_cols, dropna=False, observed=True)
    .agg(
        Passengers=("Passengers", "sum"),
        Seats=("Seats", "sum"),
        DepScheduled=("DepScheduled", "sum"),
        DepPerformed=("DepPerformed", "sum"),
        DistanceMin=("Distance", "min"),
        DistanceMax=("Distance", "max"),
        SegmentRows=("Passengers", "size"),
    )
    .reset_index()
)
return agg

def ingest_and_aggregate(
    archives: list[Path],
    chunksize: int = 200_000,
) -> tuple[pd.DataFrame, dict[str, object], pd.DataFrame]:
    """Stream annual archives and build a memory-efficient carrier-route-month table.

    Function collects audit metrics for selected T-100 rows. Only will keep Service
    Class F record for project core scheduled-passenger table.
    """
    partials: list[pd.DataFrame] = []
    total_rows = 0
    scheduled_rows = 0
    class_counts: dict[str, int] = {}
    source_years: set[int] = set()
    source_columns: set[str] = set()

    for zip_path in archives:
        member = _data_member(zip_path)
        with ZipFile(zip_path) as zf, zf.open(member) as fh:
            header = pd.read_csv(fh, nrows=0)
            resolved = _resolve_columns(header.columns)
            source_columns.update(resolved.keys())
            usecols = list(resolved.values())
            rename_map = {actual: canonical for canonical, actual in resolved.items()}

            with ZipFile(zip_path) as zf, zf.open(member) as fh:
                reader = pd.read_csv(
                    fh,
                    usecols=usecols,
                    chunksize=chunksize,
                    low_memory=False,
                )
                for raw_chunk in reader:
                    chunk = _coerce_types(raw_chunk.rename(columns=rename_map))
                    total_rows += len(chunk)
                    if "Year" in chunk:
                        source_years.update(chunk["Year"].dropna().astype(int).unique().tolist())

                    cls = chunk["Class"].fillna("<MISSING>")
                    vc = cls.value_counts(dropna=False)
                    for label, count in vc.items():
                        class_counts[str(label)] = class_counts.get(str(label), 0) + int(count)

                    scheduled = chunk.loc[chunk["Class"].str.upper().eq("F")].copy()
                    scheduled_rows += len(scheduled)
                    if not scheduled.empty:
                        partials.append(_partial_monthly_aggregate(scheduled))

    if not partials:
        raise ValueError("No Service Class F rows were found in the supplied archives.")

    combined = pd.concat(partials, ignore_index=True)
    keys = ["Year", "Month", "UniqueCarrier", "Origin", "Dest"]
    id_candidates = [
        "AirlineID",
        "UniqueCarrierName",
        "OriginAirportID",
        "OriginCityName",
        "OriginState",
        "DestAirportID",
        "DestCityName",
        "DestState",
    ]
    ids = [c for c in id_candidates if c in combined.columns]

    # Aggregate chunks.
    # Numeric identifiers/names stable in carrier-route-month.
    # First non-null value retained for descriptive stats.
    agg_spec: dict[str, tuple[str, str]] = {
        "Passengers": ("Passengers", "sum"),

```

```

        "Seats": ("Seats", "sum"),
        "DepScheduled": ("DepScheduled", "sum"),
        "DepPerformed": ("DepPerformed", "sum"),
        "DistanceMin": ("DistanceMin", "min"),
        "DistanceMax": ("DistanceMax", "max"),
        "SegmentRows": ("SegmentRows", "sum"),
    }
    for col in ids:
        agg_spec[col] = (col, "first")

    monthly = (
        combined.groupby(keys, dropna=False, observed=True)
        .agg(**agg_spec)
        .reset_index()
        .sort_values(keys)
        .reset_index(drop=True)
    )
    monthly["Quarter"] = ((monthly["Month"] - 1) // 3 + 1).astype("Int64")
    monthly["Period"] = pd.to_datetime(
        dict(year=monthly["Year"].astype(int), month=monthly["Month"].astype(int), day=1),
        errors="coerce",
    )
    monthly["Route"] = monthly["Origin"].astype(str) + "→" + monthly["Dest"].astype(str)
    monthly["Distance"] = monthly["DistanceMax"]
    monthly["LoadFactor"] = np.where(
        monthly["Seats"] > 0, monthly["Passengers"] / monthly["Seats"], np.nan
    )

    # Construct the t+1 passenger target by an exact one-calendar-month self-join.
    series_keys = ["UniqueCarrier", "Origin", "Dest"]
    target = monthly[series_keys + ["Period", "Passengers"]].copy()
    target["Period"] = target["Period"] - pd.DateOffset(months=1)
    target = target.rename(columns={"Passengers": "TargetPassengersNextMonth"})
    monthly = monthly.merge(
        target[series_keys + ["Period", "TargetPassengersNextMonth"]],
        on=series_keys + ["Period"],
        how="left",
        validate="one_to_one",
    )

    class_distribution = (
        pd.DataFrame(
            {"Class": list(class_counts.keys()), "Rows": list(class_counts.values())}
        )
        .sort_values("Rows", ascending=False)
        .reset_index(drop=True)
    )
    class_distribution["Percent"] = (
        100 * class_distribution["Rows"] / class_distribution["Rows"].sum()
    )

    audit = {
        "archives": len(archives),
        "selected_raw_rows": total_rows,
        "scheduled_class_f_rows": scheduled_rows,
        "source_years": sorted(source_years),
        "resolved_fields": sorted(source_columns),
    }
    return monthly, audit, class_distribution

def build_variable_profile(monthly: pd.DataFrame) -> pd.DataFrame:
    """Create target/predict table."""
    roles = {
        "TargetPassengersNextMonth": (
            "Target",
            "Next-month passenger count for the same carrier and directional route.",
        ),
    }
    for col, rationale in CANDIDATE_PREDICTOR_RATIONALE.items():
        roles[col] = ("Candidate predictor", rationale)

    records = []
    for col, (role, rationale) in roles.items():
        if col in monthly.columns:
            records.append(
                {
                    "Variable": col,
                    "Role": role,
                    "DType": str(monthly[col].dtype),
                    "NonMissing": int(monthly[col].notna().sum()),
                    "UniqueValues": int(monthly[col].nunique(dropna=True)),
                    "Definition_or_Rationale": rationale,
                    "BTS_Definition": BTS_DESCRIPTIONS.get(col, "Derived/project-defined field"),
                }
            )
    return pd.DataFrame(records)

def build_results(monthly: pd.DataFrame, audit: dict[str, object], class_distribution: pd.DataFrame) -> Week2Results:
    """Build exploration tables."""
    target_nonmissing = int(monthly["TargetPassengersNextMonth"].notna().sum())
    profile_rows = [
        ("Annual ZIP archives", int(audit["archives"])),
    ]

```

```

        ("Selected raw T-100 rows ingested", int(audit["selected_raw_rows"])),
        ("Service Class F rows", int(audit["scheduled_class_f_rows"])),
        ("Carrier-route-month rows", len(monthly)),
        ("Carrier-route-month columns", monthly.shape[1]),
        ("Unique carriers", monthly["UniqueCarrier"].nunique()),
        ("Unique origins", monthly["Origin"].nunique()),
        ("Unique destinations", monthly["Dest"].nunique()),
        ("Unique directional routes", monthly["Route"].nunique()),
        (
            "Unique carrier-route series",
            monthly[["UniqueCarrier", "Origin", "Dest"]].drop_duplicates().shape[0],
        ),
        ("Rows with next-month target", target_nonmissing),
        ("Rows without next-month target", len(monthly) - target_nonmissing),
    ]
    dataset_profile = pd.DataFrame(profile_rows, columns=["Metric", "Value"])

    missingness = pd.DataFrame(
        {
            "Variable": monthly.columns,
            "MissingCount": monthly.isna().sum().values,
            "MissingPercent": (100 * monthly.isna().mean()).values,
            "DType": monthly.dtypes.astype(str).values,
        }
    ).sort_values(["MissingPercent", "Variable"], ascending=[False, True])

    numeric_cols = [
        c
        for c in [
            "Passengers",
            "TargetPassengersNextMonth",
            "Seats",
            "DepScheduled",
            "DepPerformed",
            "Distance",
            "LoadFactor",
            "SegmentRows",
        ]
        if c in monthly.columns
    ]
    numeric_summary = monthly[numeric_cols].describe().T.reset_index().rename(columns={"index": "Variable"})

    variable_profile = build_variable_profile(monthly)

    annual_summary = (
        monthly.groupby("Year", as_index=False)
        .agg(
            CarrierRouteMonths=("Passengers", "size"),
            Passengers=("Passengers", "sum"),
            Seats=("Seats", "sum"),
            DeparturesPerformed=("DepPerformed", "sum"),
        )
        .sort_values("Year")
    )
    annual_summary["LoadFactor"] = np.where(
        annual_summary["Seats"] > 0,
        annual_summary["Passengers"] / annual_summary["Seats"],
        np.nan,
    )

    top_routes = (
        monthly.groupby("Route", as_index=False)
        .agg(Passengers=("Passengers", "sum"), CarrierRouteMonths=("Passengers", "size"))
        .sort_values("Passengers", ascending=False)
        .head(15)
        .reset_index(drop=True)
    )

    return Week2Results(
        monthly=monthly,
        dataset_profile=dataset_profile,
        class_distribution=class_distribution,
        missingness=missingness,
        numeric_summary=numeric_summary,
        variable_profile=variable_profile,
        annual_summary=annual_summary,
        top_routes=top_routes,
    )

def save_outputs(results: Week2Results, reports_dir: Path) -> None:
    """Persist report-ready tables and figures without modifying raw data."""
    tables_dir = reports_dir / "tables_Week2"
    figures_dir = reports_dir / "figures_Week2"
    tables_dir.mkdir(parents=True, exist_ok=True)
    figures_dir.mkdir(parents=True, exist_ok=True)

    results.dataset_profile.to_csv(tables_dir / "week2_output_dataset_profile.csv", index=False)
    results.class_distribution.to_csv(tables_dir / "week2_output_class_distribution.csv", index=False)
    results.missingness.to_csv(tables_dir / "week2_output_missingness.csv", index=False)
    results.numeric_summary.to_csv(tables_dir / "week2_output_numeric_summary.csv", index=False)
    results.variable_profile.to_csv(tables_dir / "week2_output_variable_profile.csv", index=False)
    results.annual_summary.to_csv(tables_dir / "week2_output_annual_summary.csv", index=False)
    results.top_routes.to_csv(tables_dir / "week2_output_top_routes.csv", index=False)

```

```

# Figure 1: annual passenger volume.
# Note final year could be be partial.
fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(results.annual_summary["Year"], results.annual_summary["Passengers"], marker="o")
ax.set_title("Scheduled-Passenger Volume by Year")
ax.set_xlabel("Year")
ax.set_ylabel("Passengers")
ax.grid(alpha=0.25)
fig.tight_layout()
fig.savefig(figures_dir / "week2_figure_annual_passengers.png", dpi=180, bbox_inches="tight")
plt.close(fig)

# Figure 2: next-month target distribution.
# Use log1p becaus route-month demand is right-skewed.
target = results.monthly["TargetPassengersNextMonth"].dropna()
fig, ax = plt.subplots(figsize=(9, 5))
ax.hist(np.log1p(target), bins=50)
ax.set_title("Distribution of Next-Month Passenger Target")
ax.set_xlabel("log(1 + next-month passengers)")
ax.set_ylabel("Carrier-route-month observations")
fig.tight_layout()
fig.savefig(figures_dir / "week2_figure_target_distribution_log.png", dpi=180, bbox_inches="tight")
plt.close(fig)

# Figure 3: top directional routes by passengers across exploration time.
plot_data = results.top_routes.sort_values("Passengers")
fig, ax = plt.subplots(figsize=(9, 6))
ax.barh(plot_data["Route"], plot_data["Passengers"])
ax.set_title("Top Directional Routes by Passenger Volume")
ax.set_xlabel("Passengers")
ax.set_ylabel("Directional route")
fig.tight_layout()
fig.savefig(figures_dir / "week2_figure_top_routes.png", dpi=180, bbox_inches="tight")
plt.close(fig)

def run_week2(project_root: Path | None = None, chunksize: int = 200_000) -> Week2Results:
    """Run ingestion/exploration workflow."""
    root = project_root or detect_project_root()
    raw_dir = root / "Data" / "raw"
    reports_dir = root / "Reports"
    archives = discover_archives(raw_dir)
    monthly, audit, class_distribution = ingest_and_aggregate(archives, chunksize=chunksize)
    results = build_results(monthly, audit, class_distribution)
    save_outputs(results, reports_dir)
    return results

if __name__ == "__main__":
    root = detect_project_root()
    results = run_week2(root)
    print("Ingestion / exploration complete.")
    print(results.dataset_profile.to_string(index=False))

```

```

Ingestion / exploration complete.
      Metric  Value
Annual ZIP archives      12
Selected raw T-100 rows ingested 4512207
Service Class F rows 3396473
Carrier-route-month rows 1882779
Carrier-route-month columns      24
Unique carriers      181
Unique origins      1006
Unique destinations  1030
Unique directional routes  35790
Unique carrier-route series  96545
Rows with next-month target 1566289
Rows without next-month target 316490

```

2. Find annual BTS data

```

In [3]: archives = discover_archives(RAW_DIR)
print(f"Annual ZIP archives found: {len(archives)}")
for path in archives:
    print(f"- {path.name}: {path.stat().st_size / 1024**2:.2f} MiB")

assert len(archives) >= 1, "At least one raw BTS archive is required."

```

```

Annual ZIP archives found: 12
- week1_data_t100_domestic_segment_all_carriers_2015.zip: 11.28 MiB
- week1_data_t100_domestic_segment_all_carriers_2016.zip: 11.50 MiB
- week1_data_t100_domestic_segment_all_carriers_2017.zip: 11.47 MiB
- week1_data_t100_domestic_segment_all_carriers_2018.zip: 11.57 MiB
- week1_data_t100_domestic_segment_all_carriers_2019.zip: 12.03 MiB
- week1_data_t100_domestic_segment_all_carriers_2020.zip: 9.26 MiB
- week1_data_t100_domestic_segment_all_carriers_2021.zip: 11.67 MiB
- week1_data_t100_domestic_segment_all_carriers_2022.zip: 12.57 MiB
- week1_data_t100_domestic_segment_all_carriers_2023.zip: 12.75 MiB
- week1_data_t100_domestic_segment_all_carriers_2024.zip: 13.05 MiB
- week1_data_t100_domestic_segment_all_carriers_2025.zip: 13.87 MiB
- week1_data_t100_domestic_segment_all_carriers_2026.zip: 6.01 MiB

```

3. Inspect source schema before loading data

```
In [4]: member_name, schema_map = inspect_archive_schema(archives[0])
print(f"Data member selected inside first ZIP: {member_name}")
display(schema_map)
```

Data member selected inside first ZIP: T_T100D_SEGMENT_ALL_CARRIER.csv

	CanonicalField	SourceHeader
0	Year	YEAR
1	Quarter	QUARTER
2	Month	MONTH
3	UniqueCarrier	UNIQUE_CARRIER
4	AirlineID	AIRLINE_ID
5	UniqueCarrierName	UNIQUE_CARRIER_NAME
6	OriginAirportID	ORIGIN_AIRPORT_ID
7	Origin	ORIGIN
8	OriginCityName	ORIGIN_CITY_NAME
9	DestAirportID	DEST_AIRPORT_ID
10	Dest	DEST
11	DestCityName	DEST_CITY_NAME
12	AircraftType	AIRCRAFT_TYPE
13	AircraftConfig	AIRCRAFT_CONFIG
14	Class	CLASS
15	Passengers	PASSENGERS
16	Seats	SEATS
17	DepScheduled	DEPARTURES_SCHEDULED
18	DepPerformed	DEPARTURES_PERFORMED
19	Distance	DISTANCE

4. Ingest / aggregate dataset

```
In [5]: monthly, audit, class_distribution = ingest_and_aggregate(
        archives,
        chunksize=200_000,
    )
results = build_results(monthly, audit, class_distribution)
save_outputs(results, REPORTS_DIR)

print("Ingestion and Week 2 exploration tables completed successfully.")
```

Ingestion and Week 2 exploration tables completed successfully.

5. General dataset statistics

```
In [6]: display(results.dataset_profile)

print("\nTemporal coverage:")
print(f"Start: {monthly['Period'].min():%Y-%m}")
print(f"End:   {monthly['Period'].max():%Y-%m}")
print(f"Shape of carrier-route-month table: {monthly.shape}")
```

	Metric	Value
0	Annual ZIP archives	12
1	Selected raw T-100 rows ingested	4512207
2	Service Class F rows	3396473
3	Carrier-route-month rows	1882779
4	Carrier-route-month columns	24
5	Unique carriers	181
6	Unique origins	1006
7	Unique destinations	1030
8	Unique directional routes	35790
9	Unique carrier-route series	96545
10	Rows with next-month target	1566289
11	Rows without next-month target	316490

Temporal coverage:
Start: 2015-01
End: 2026-05
Shape of carrier-route-month table: (1882779, 24)

6. Service-class distribution

The download is the **All Carriers** T-100 segment table and can contain multiple service classes.

```
In [7]: display(results.class_distribution)
```

	Class	Rows	Percent
0	F	3396473	75.272987
1	L	556375	12.330441
2	G	433452	9.606208
3	P	125907	2.790364

7. Preview table

```
In [8]: preview_cols = [
    "Period", "UniqueCarrier", "Origin", "Dest", "Route",
    "Passengers", "TargetPassengersNextMonth", "Seats",
    "DepScheduled", "DepPerformed", "Distance", "LoadFactor"
]
display(monthly[preview_cols].head(15))
```

	Period	UniqueCarrier	Origin	Dest	Route	Passengers	TargetPassengersNextMonth	Seats	DepScheduled	DepPerformed	Distance	LoadFactor
0	2015-01-01	0JQ	GCN	LAS	GCN→LAS	28.0	409.0	136.0	1.0	1.0	169.0	0.205882
1	2015-01-01	0JQ	LAS	GCN	LAS→GCN	28.0	386.0	136.0	1.0	1.0	169.0	0.205882
2	2015-01-01	1DQ	ACK	HYA	ACK→HYA	2348.0	2530.0	3042.0	360.0	338.0	31.0	0.771861
3	2015-01-01	1DQ	HYA	ACK	HYA→ACK	2288.0	2392.0	3168.0	356.0	352.0	31.0	0.722222
4	2015-01-01	1PQ	HNL	LUP	HNL→LUP	164.0	161.0	486.0	52.0	54.0	61.0	0.337449
5	2015-01-01	1PQ	LUP	HNL	LUP→HNL	182.0	164.0	477.0	51.0	53.0	61.0	0.381551
6	2015-01-01	1PQ	LUP	MKK	LUP→MKK	249.0	246.0	711.0	78.0	79.0	9.0	0.350211
7	2015-01-01	1PQ	MKK	LUP	MKK→LUP	200.0	195.0	702.0	78.0	78.0	9.0	0.284900
8	2015-01-01	1QQ	STT	STX	STT→STX	1047.0	1160.0	1584.0	176.0	176.0	45.0	0.660985
9	2015-01-01	1QQ	STX	STT	STX→STT	1295.0	1003.0	1584.0	176.0	176.0	45.0	0.817551
10	2015-01-01	1WQ	CLT	LUK	CLT→LUK	400.0	362.0	600.0	20.0	20.0	330.0	0.666667
11	2015-01-01	1WQ	CVG	MDW	CVG→MDW	125.0	142.0	600.0	20.0	20.0	249.0	0.208333
12	2015-01-01	1WQ	CVG	MMU	CVG→MMU	260.0	287.0	420.0	14.0	14.0	558.0	0.619048
13	2015-01-01	1WQ	LUK	CLT	LUK→CLT	408.0	380.0	600.0	20.0	20.0	330.0	0.680000
14	2015-01-01	1WQ	LUK	MDW	LUK→MDW	368.0	399.0	600.0	20.0	20.0	255.0	0.613333

8. Target variable and candidate predictors

Target: TargetPassengersNextMonth — passenger volume in month $t+1$ for the same carrier and directional route.

```
In [9]: display(results.variable_profile)
```

	Variable	Role	DType	NonMissing	UniqueValues	Definition_or_Rationale	BTS_Definition
0	TargetPassengersNextMonth	Target	float64	1566289	49411	Next-month passenger count for the same carrie...	Derived/project-defined field
1	Passengers	Candidate predictor	float64	1882779	49477	Most recent observed demand at forecast origin...	Non-stop segment passengers transported
2	Seats	Candidate predictor	float64	1882779	56317	Most recent available capacity; proxy for plan...	Available seats
3	DepScheduled	Candidate predictor	float64	1882779	946	Most recent scheduled frequency; captures offe...	Departures scheduled
4	DepPerformed	Candidate predictor	float64	1882779	877	Most recent realized frequency; captures actua...	Departures performed
5	LoadFactor	Candidate predictor	float64	1878356	906563	Capacity utilization measure; may indicate sup...	Ratio of passenger miles to available seat miles
6	Distance	Candidate predictor	float64	1882779	2586	Stable route characteristic associated with ma...	Distance between airports (miles)
7	UniqueCarrier	Candidate predictor	string	1882742	181	Carrier-specific network, fleet, and commercia...	Unique carrier code; preferred for analysis ac...
8	Origin	Candidate predictor	string	1882779	1006	Origin-market identity; captures location-spec...	Origin airport code
9	Dest	Candidate predictor	string	1882779	1030	Destination-market identity; captures location...	Destination airport code
10	Year	Candidate predictor	int64	1882779	12	Long-run trend/regime information.	Calendar year
11	Quarter	Candidate predictor	Int64	1882779	4	Broad seasonal pattern.	Calendar quarter
12	Month	Candidate predictor	int64	1882779	12	Fine-grained calendar seasonality.	Calendar month

9. Data types and missingness

```
In [10]: display(results.missingness)

print("Columns with any missing values:")
display(results.missingness.loc[results.missingness["MissingCount"] > 0])
```

	Variable	MissingCount	MissingPercent	DType
23	TargetPassengersNextMonth	316490	16.809726	float64
22	LoadFactor	4423	0.234919	float64
12	AirlineID	37	0.001965	float64
2	UniqueCarrier	37	0.001965	string
13	UniqueCarrierName	37	0.001965	string
8	DepPerformed	0	0.000000	float64
7	DepScheduled	0	0.000000	float64
4	Dest	0	0.000000	string
16	DestAirportID	0	0.000000	int64
17	DestCityName	0	0.000000	string
21	Distance	0	0.000000	float64
10	DistanceMax	0	0.000000	float64
9	DistanceMin	0	0.000000	float64
1	Month	0	0.000000	int64
3	Origin	0	0.000000	string
14	OriginAirportID	0	0.000000	int64
15	OriginCityName	0	0.000000	string
5	Passengers	0	0.000000	float64
19	Period	0	0.000000	datetime64[ns]
18	Quarter	0	0.000000	Int64
20	Route	0	0.000000	object
6	Seats	0	0.000000	float64
11	SegmentRows	0	0.000000	int64
0	Year	0	0.000000	int64

Columns with any missing values:

	Variable	MissingCount	MissingPercent	DType
23	TargetPassengersNextMonth	316490	16.809726	float64
22	LoadFactor	4423	0.234919	float64
12	AirlineID	37	0.001965	float64
2	UniqueCarrier	37	0.001965	string
13	UniqueCarrierName	37	0.001965	string

10. Numeric descriptive statistics

In [11]: `display(results.numeric_summary)`

	Variable	count	mean	std	min	25%	50%	75%	max
0	Passengers	1882779.0	4441.502472	7394.087847	0.0	198.000000	1865.000000	4934.000000	98532.00
1	TargetPassengersNextMonth	1566289.0	5231.532678	7831.190558	0.0	793.000000	2657.000000	5981.000000	98532.00
2	Seats	1882779.0	5585.423784	8904.585572	0.0	333.000000	2434.000000	6196.000000	114935.00
3	DepScheduled	1882779.0	47.027044	61.935034	0.0	5.000000	30.000000	62.000000	2605.00
4	DepPerformed	1882779.0	46.988528	59.363012	0.0	8.000000	29.000000	61.000000	1576.00
5	Distance	1882779.0	720.711325	608.455619	0.0	266.000000	588.000000	997.000000	6089.00
6	LoadFactor	1878356.0	0.700271	0.254046	0.0	0.614035	0.791148	0.876928	3.25
7	SegmentRows	1882779.0	1.803968	1.186173	1.0	1.000000	1.000000	2.000000	18.00

11. Categorical/cardinality exploration

```
In [12]: categorical_summary = pd.DataFrame({
    "Variable": ["UniqueCarrier", "Origin", "Dest", "Route"],
    "UniqueValues": [
        monthly["UniqueCarrier"].nunique(),
        monthly["Origin"].nunique(),
        monthly["Dest"].nunique(),
        monthly["Route"].nunique(),
    ],
})
display(categorical_summary)

print("Top carriers by carrier-route-month observations:")
display(monthly["UniqueCarrier"].value_counts().head(15).rename_axis("UniqueCarrier").reset_index(name="Rows"))

print("Top directional routes by total passengers:")
display(results.top_routes)
```

	Variable	UniqueValues
0	UniqueCarrier	181
1	Origin	1006
2	Dest	1030
3	Route	35790

Top carriers by carrier-route-month observations:

	UniqueCarrier	Rows
0	WN	213189
1	OO	198767
2	DL	126231
3	AA	120296
4	UA	103930
5	G4	103335
6	YX	83713
7	MQ	66157
8	F9	64336
9	9E	59534
10	NK	59245
11	B6	53609
12	EV	51863
13	OH	50500
14	AS	44921

Top directional routes by total passengers:

	Route	Passengers	CarrierRouteMonths
0	LAX→JFK	16790310.0	599
1	JFK→LAX	16671868.0	600
2	LAX→SFO	16389898.0	954
3	SFO→LAX	16218378.0	952
4	ATL→MCO	15208964.0	623
5	MCO→ATL	15111509.0	599
6	LAS→LAX	14424091.0	1205
7	LAX→LAS	14229112.0	1173
8	LGA→ORD	13992657.0	762
9	ORD→LGA	13981696.0	764
10	LAX→ORD	13667546.0	578
11	ORD→LAX	13476129.0	559
12	FLL→ATL	13414433.0	544
13	ATL→FLL	13320945.0	557
14	PHX→DEN	12368729.0	712

12. Temporal exploration

```
In [13]: display(results.annual_summary)
```

	Year	CarrierRouteMonths	Passengers	Seats	DeparturesPerformed	LoadFactor
0	2015	163939	705548135.0	8.545951e+08	8061159.0	0.825594
1	2016	168525	728843342.0	8.921574e+08	8185533.0	0.816945
2	2017	163638	750307963.0	9.176742e+08	8177634.0	0.817619
3	2018	168175	785793374.0	9.588728e+08	8399975.0	0.819497
4	2019	174066	818676252.0	9.926802e+08	8596713.0	0.824713
5	2020	129041	340279776.0	5.987654e+08	5213008.0	0.568302
6	2021	154990	611032239.0	8.031037e+08	6759313.0	0.760838
7	2022	161899	757310329.0	9.253256e+08	7423937.0	0.818426
8	2023	166343	826030895.0	1.018910e+09	7808323.0	0.810700
9	2024	173928	856508236.0	1.054405e+09	8137678.0	0.812314
10	2025	182575	845236869.0	1.066498e+09	8334966.0	0.792535
11	2026	75660	336800172.0	4.331307e+08	3370775.0	0.777595

13. Week 2 figures

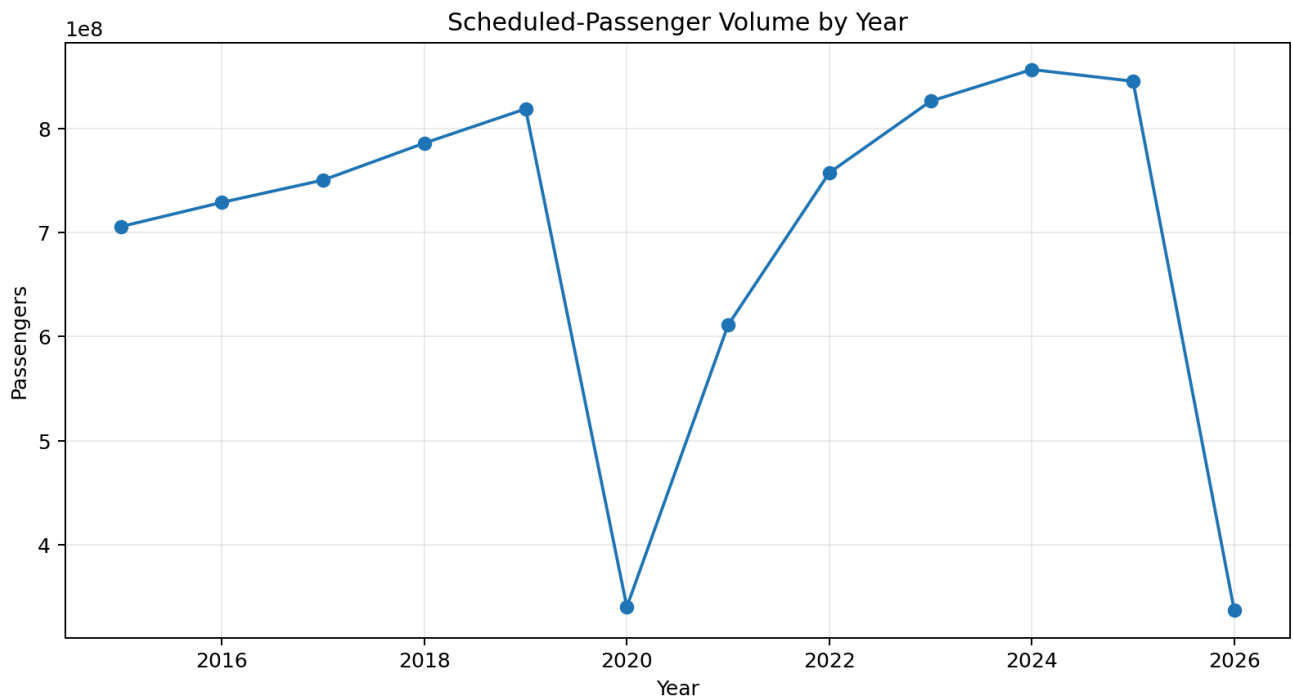
Save three figures to `Reports/figures_Week2/` :

1. scheduled-passenger volume by year,
2. the next-month passenger target distribution on a `log(1+x)` scale,
3. top directional routes by passenger volume.

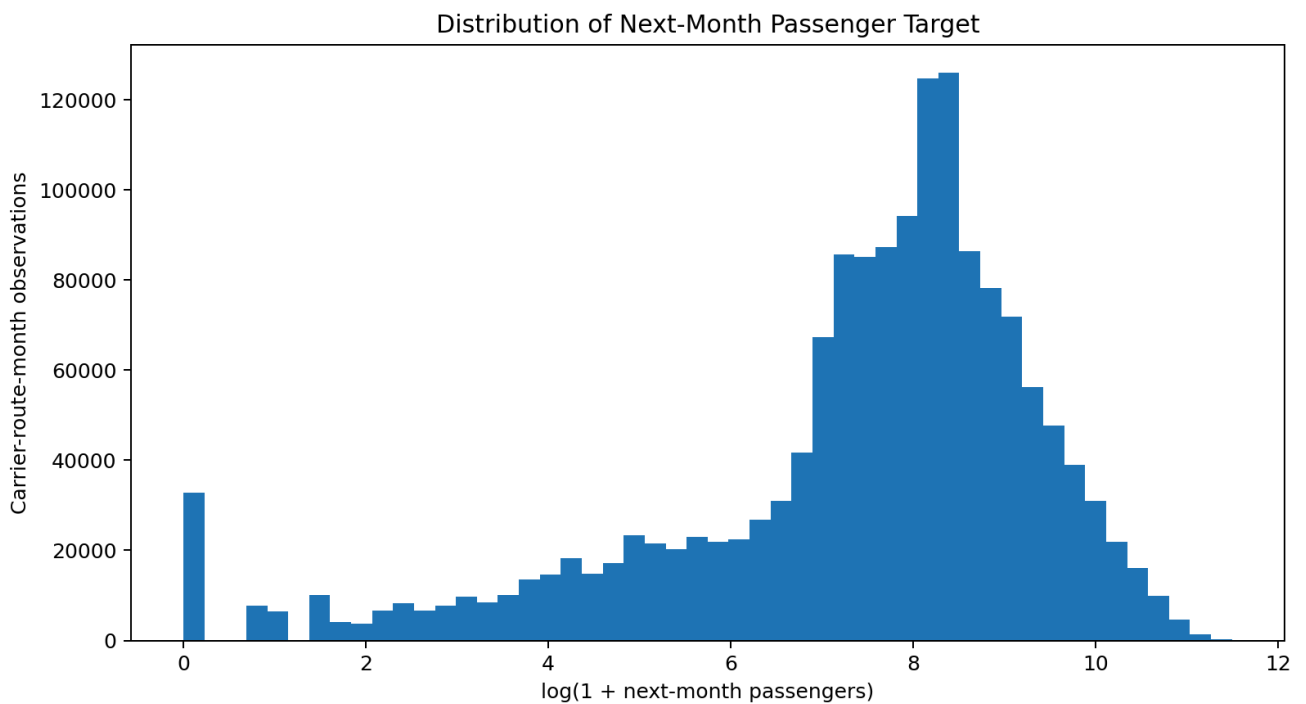
```
In [14]: from IPython.display import Image, display as ipy_display

figure_paths = [
    REPORTS_DIR / "figures_Week2" / "week2_figure_annual_passengers.png",
    REPORTS_DIR / "figures_Week2" / "week2_figure_target_distribution_log.png",
    REPORTS_DIR / "figures_Week2" / "week2_figure_top_routes.png",
]
for fig_path in figure_paths:
    print(fig_path.name)
    ipy_display(Image(filename=str(fig_path)))
```

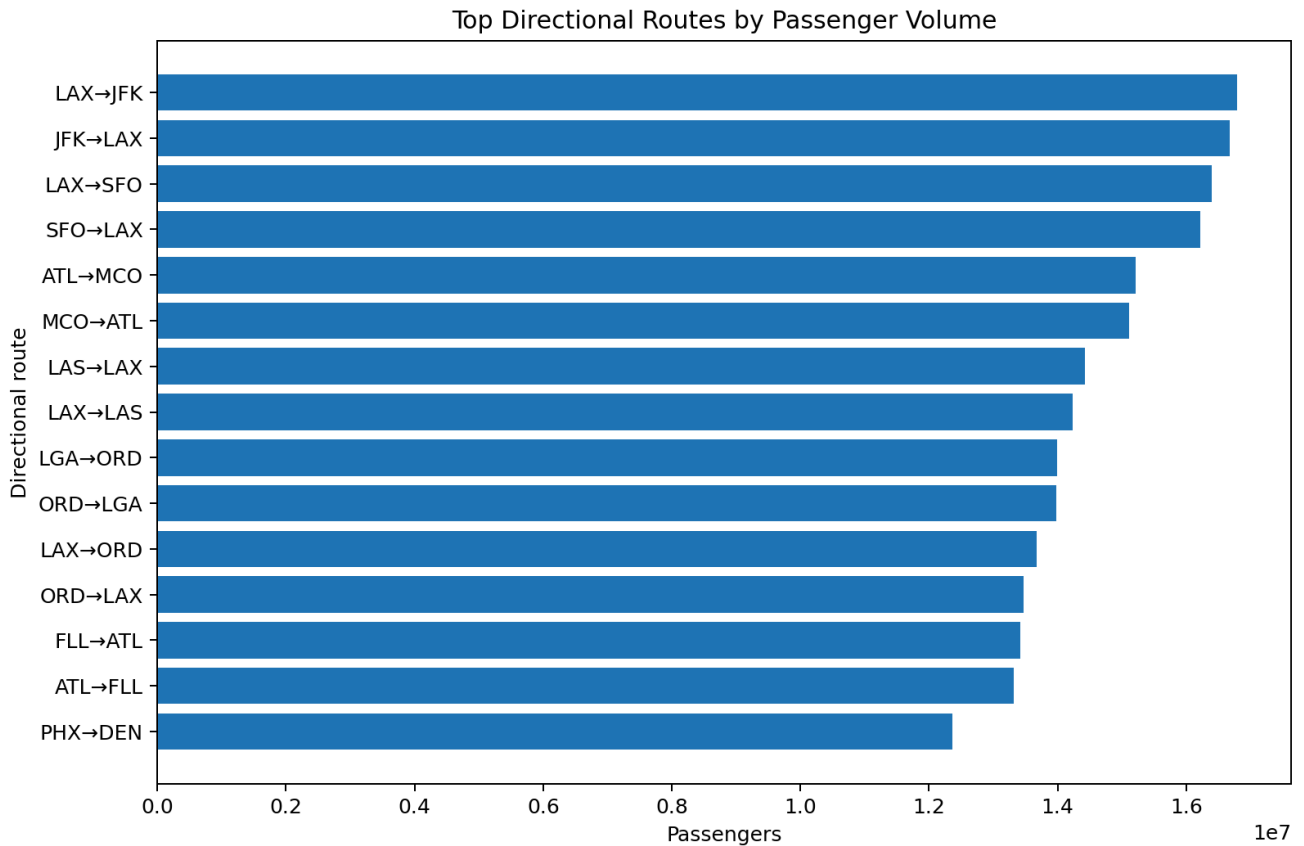
week2_figure_annual_passengers.png



week2_figure_target_distribution_log.png



week2_figure_top_routes.png



14. Reasonableness / reproducibility checks

```
In [15]: assert not monthly.empty, "Analytical table is unexpectedly empty."
assert monthly["Month"].dropna().between(1, 12).all(), "Invalid month values detected."
assert (monthly["Passengers"].dropna() >= 0).all(), "Negative passenger counts detected."
assert (monthly["Seats"].dropna() >= 0).all(), "Negative seat counts detected."
assert monthly["TargetPassengersNextMonth"].notna().any(), "No next-month targets were constructed."
assert monthly[["UniqueCarrier", "Origin", "Dest", "Period"]].duplicated().sum() == 0, "Duplicate analytical keys detected."

print("All Week 2 reasonableness checks passed.")
```

All Week 2 reasonableness checks passed.

15. Saved Week 2 outputs

```
In [16]: saved_tables = sorted((REPORTS_DIR / "tables_Week2").glob("week2_output_*.csv"))
saved_figures = sorted((REPORTS_DIR / "figures_Week2").glob("week2_figure_*.png"))

print("Saved tables:")
for p in saved_tables:
    print(f"- {p.relative_to(PROJECT_ROOT)}")

print("\nSaved figures:")
for p in saved_figures:
    print(f"- {p.relative_to(PROJECT_ROOT)}")
```

Saved tables:

- Reports\tables_Week2\week2_output_annual_summary.csv
- Reports\tables_Week2\week2_output_class_distribution.csv
- Reports\tables_Week2\week2_output_dataset_profile.csv
- Reports\tables_Week2\week2_output_missingness.csv
- Reports\tables_Week2\week2_output_numeric_summary.csv
- Reports\tables_Week2\week2_output_top_routes.csv
- Reports\tables_Week2\week2_output_variable_profile.csv

Saved figures:

- Reports\figures_Week2\week2_figure_annual_passengers.png
- Reports\figures_Week2\week2_figure_target_distribution_log.png
- Reports\figures_Week2\week2_figure_top_routes.png

===== WEEK 2 END =====